

Introducción a la Ingeniería de Software

De por qué saber programar no alcanza.

UTN • Facultad Regional Córdoba
Cátedra de Ingeniería y Calidad de Software
Docentes: Judith Meles – Laura Covaro
Elaborado a partir de la presentación «Ingeniería de Software» (48 láminas)

CONTENIDO

- 1. Un poco de historia
- 2. El software mueve la economía
- 3. Los problemas de siempre
- 4. Qué es el software
- 5. Software no es manufactura
- 6. No hay balas de plata: esencia y accidente
- 7. Qué dicen los datos: éxito y fracaso
- 8. Qué es la Ingeniería de Software
- 9. SWEBOK y SEBoK: los cuerpos de conocimiento
- 10. El proceso de software
- 11. Procesos definidos vs. empíricos
- 12. Ciclos de vida
- 13. Preguntas de repaso
- 14. Bibliografía

01 Un poco de historia

Lám. 2

La disciplina nace de una crisis. Para fines de los años 60 el hardware avanzaba mucho más rápido que la capacidad de construir el software que corría sobre él: los proyectos se atrasaban, se pasaban de presupuesto y entregaban sistemas poco confiables. En la **conferencia de la OTAN de 1968** se acuñó deliberadamente el término «ingeniería de software», como provocación: si construir software se parecía tan poco a una ingeniería, había que convertirlo en una.

AÑO	HITO
1968	Nace el término, en la conferencia de la NATO.
1975	<i>The Mythical Man-Month</i> , de Frederick Brooks. De ahí sale la idea de que agregar gente a un proyecto atrasado lo atrasa todavía más.
1978	Tom DeMarco introduce el <i>Structured Analysis</i> .
Años 80	Se hacen públicos los primeros grandes errores de software con consecuencias graves.

1987	<i>No Silver Bullet</i> (Brooks): las características esenciales del software.
1989	<i>Managing the Software Process</i> , de Watts Humphrey: el foco se corre al proceso.
1990	Internet y el paradigma de objetos cambian la escala y la forma de construir.
1991 / 1993	CMM 1.0 y CMM 1.1: modelos de madurez de procesos.
2000	CMMI 1.0.
2001	Manifiesto Ágil.
2003	<i>Lean Software Development</i> .

Leída de corrido, la línea de tiempo cuenta un movimiento pendular: primero se intentó domar el software con **estructura y método** (análisis estructurado), después con **madurez de proceso** (CMM/CMMI) y finalmente con **adaptación y feedback corto** (ágil, lean). Ninguna etapa reemplaza del todo a la anterior; conviven según el tipo de proyecto.

02 El software mueve la economía

Lám. 3–5

La presentación muestra la misma foto tomada en tres momentos distintos, y el mensaje es económico antes que técnico: las empresas más valiosas del mundo pasaron de vender petróleo, tabaco o bebidas a vender software, plataformas y chips para software.

AÑO	TOP DEL RANKING
2006	Microsoft, GE, Coca-Cola, China Mobile, Marlboro.
2012	Apple, IBM, Google, McDonald's, Microsoft.
2017	Google, Apple, Microsoft, Amazon, Facebook.
2019	Apple, Microsoft, Amazon, Alphabet, Berkshire Hathaway.
2026	NVIDIA, Alphabet, Apple, Microsoft, Amazon, TSMC, Broadcom...

De las diez mayores capitalizaciones bursátiles de 2026, ocho son empresas de tecnología, y las dos excepciones (una petrolera y una automotriz que se define como empresa de software sobre ruedas) confirman la tendencia más que contradecirla. La consecuencia para nosotros es directa: **el software dejó de ser un costo de soporte para convertirse en el producto**. Cuando el producto es el software, la calidad con la que se lo construye es un problema de negocio, no una cuestión estética de programadores.

03 Los problemas de siempre

Lám. 6–8

Los cinco síntomas típicos

- La versión final **no satisface las necesidades del cliente**.
- **No es fácil extenderlo ni adaptarlo**: agregar funcionalidad en la versión siguiente resulta casi imposible.
- **Mala documentación**.
- **Mala calidad**.
- **Más tiempo y más costo** que lo presupuestado.

Conviene notar que sólo uno de estos cinco síntomas se refiere al funcionamiento del programa. Los otros cuatro son problemas de **comunicación, diseño y gestión**. Ese desbalance es, en el fondo, todo el argumento de la materia.

Errores caros de la historia

CASO	CAUSA
Mariner I (1962)	Una fórmula transcrita mal en el software de guiado; hubo que destruir la sonda a los 293 segundos del despegue.
Hartford Coliseum (1978)	Cálculo incorrecto cargado en el software CAD con el que se diseñó el techo; se derrumbó por el peso de la nieve.
Gusano de Morris (1988)	Primer ataque masivo con gusanos en Internet.
Bug de Intel (1994)	Falla de cálculo en el procesador Pentium; hubo que sustituir chips.
Ariane 5 (1996)	Un real de 64 bits (punto flotante) asociado a la velocidad se convirtió en un entero de 16 bits: desbordamiento y explosión del cohete.
Mars Climate Orbiter (1999)	Un programa calculaba en unidades inglesas y otro en unidades métricas. Se perdió la sonda.
Frenos Toyota (2010)	Retraso introducido por software en el sistema antibloqueo; más de 400.000 vehículos híbridos retirados.
Y2K (2000)	Software que no contemplaba el cambio de siglo; se gastó una fortuna en mitigarlo.

OJO CON LAS CIFRAS

En las láminas aparece «3 billones de dólares» para Toyota y «296,7 billones» para el Y2K. Son traducciones literales del inglés *billion*: en castellano corresponden a **3.000 millones** y **296.700 millones** respectivamente. El orden de magnitud sigue siendo enorme, pero conviene no repetir el error en un informe.

Y no son historias viejas: la presentación cierra con recalls contemporáneos —1,4 millones de Jeep retirados por una vulnerabilidad que permitía tomar control del vehículo a distancia, 1,9 millones de Prius por un problema de software del sistema híbrido, 300 Subaru al desguace por una falla en los robots soldadores, aviones Airbus A350 que debían reiniciarse cada 149 horas—. **El software se volvió crítico para la seguridad física de las personas.**

04 Qué es el software

lám. 9–11

La definición operativa es sencilla: **software es un conjunto de programas más la documentación que los acompaña**. Que la documentación esté dentro de la definición no es un detalle: un ejecutable sin especificación, sin diseño y sin manual es un activo que nadie puede mantener.

Se distinguen tres tipos básicos:

- **Software de sistema** (sistemas operativos, drivers, compiladores).
- **Utilitarios** (herramientas de propósito general).
- **Software de aplicación** (el que resuelve el problema de un dominio concreto).

Una definición integradora

El software es conocimiento representado en distintos niveles de abstracción.

Pantaleo & Rinaudo [2018]

Esta definición es más útil que la anterior porque explica *por qué* construir software es difícil. Si el software es conocimiento representado, entonces el trabajo real consiste en **capturar conocimiento que vive en la cabeza de otras personas y bajarlo**,

sin perderlo, por una escalera de abstracciones hasta llegar a instrucciones que una máquina ejecute.



Lámina 10. El software como conocimiento representado en distintos niveles de abstracción: de la idea al silicio.

La escalera va de lo más abstracto a lo más físico: la idea, el diseño, el código fuente, las bibliotecas y el sistema operativo, el binario, el hardware. En cada escalón se pierde intención y se gana precisión, y cada traducción es una oportunidad de introducir un defecto.

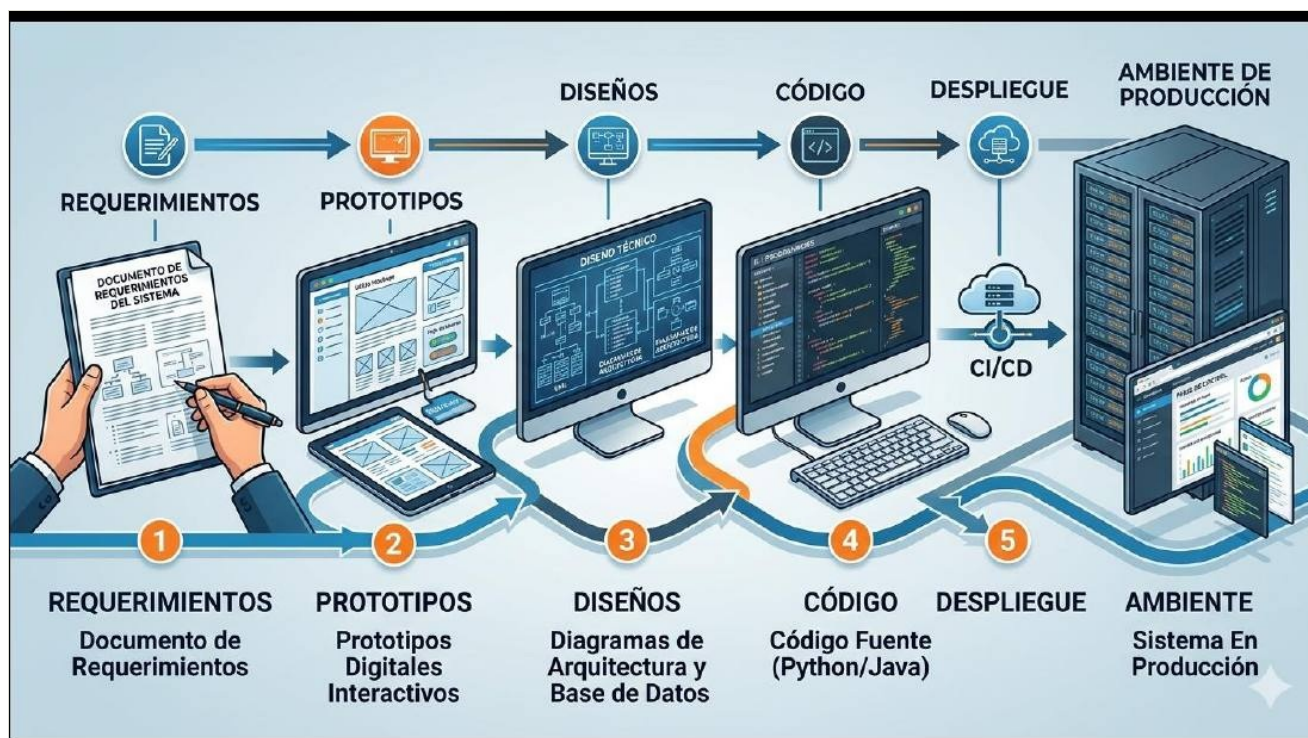


Lámina 11. La cadena de artefactos: requerimientos → prototipos → diseños → código → despliegue → producción.

La cadena de artefactos que produce un proyecto es el reflejo concreto de esa escalera: **requerimientos → prototipos → diseños → código → despliegue → producción**. Cada flecha es una transformación de conocimiento, y la trazabilidad entre eslabones es

lo que después permite responder «¿por qué el sistema hace esto?».

05 Software no es manufactura

Lám. 12

Es tentador pensar el desarrollo como una fábrica: entra materia prima, sale producto, se optimiza la línea. La analogía se rompe por cinco motivos:

1. **El software es menos predecible.** El esfuerzo depende de descubrimientos que se hacen mientras se construye.
2. **No hay producción en masa.** Casi ningún producto de software es igual a otro; copiar un binario es gratis, y por lo tanto lo único que se «produce» es siempre algo nuevo.
3. **No todas las fallas son errores.** Un sistema puede funcionar exactamente como fue especificado y aun así ser un fracaso, porque la especificación estaba equivocada.
4. **El software no se gasta.** No hay fatiga de materiales: si se degrada, es porque cambió su entorno o porque cada modificación fue dejando cicatrices.
5. **El software no está gobernado por las leyes de la física.** No hay límite natural que impida diseñar algo absurdo, y por eso la disciplina tiene que venir de las personas y del proceso.

Crear software es una tarea intelectual humana. El buen software depende de las personalidades y los intelectos de quienes integran el equipo; un proceso que le funciona de maravilla a un equipo puede no producir nada en otro.

The Practical Guide to Enterprise Architecture (paráfrasis)

06 No hay balas de plata: esencia y accidente

Lám. 13–20

En 1987 Brooks publicó *No Silver Bullet*, el ensayo más citado de la disciplina. Su tesis: **no existe ni existirá una sola técnica, lenguaje o herramienta que por sí misma produzca una mejora de un orden de magnitud en productividad, confiabilidad y simplicidad.** Para argumentarlo toma una distinción aristotélica.

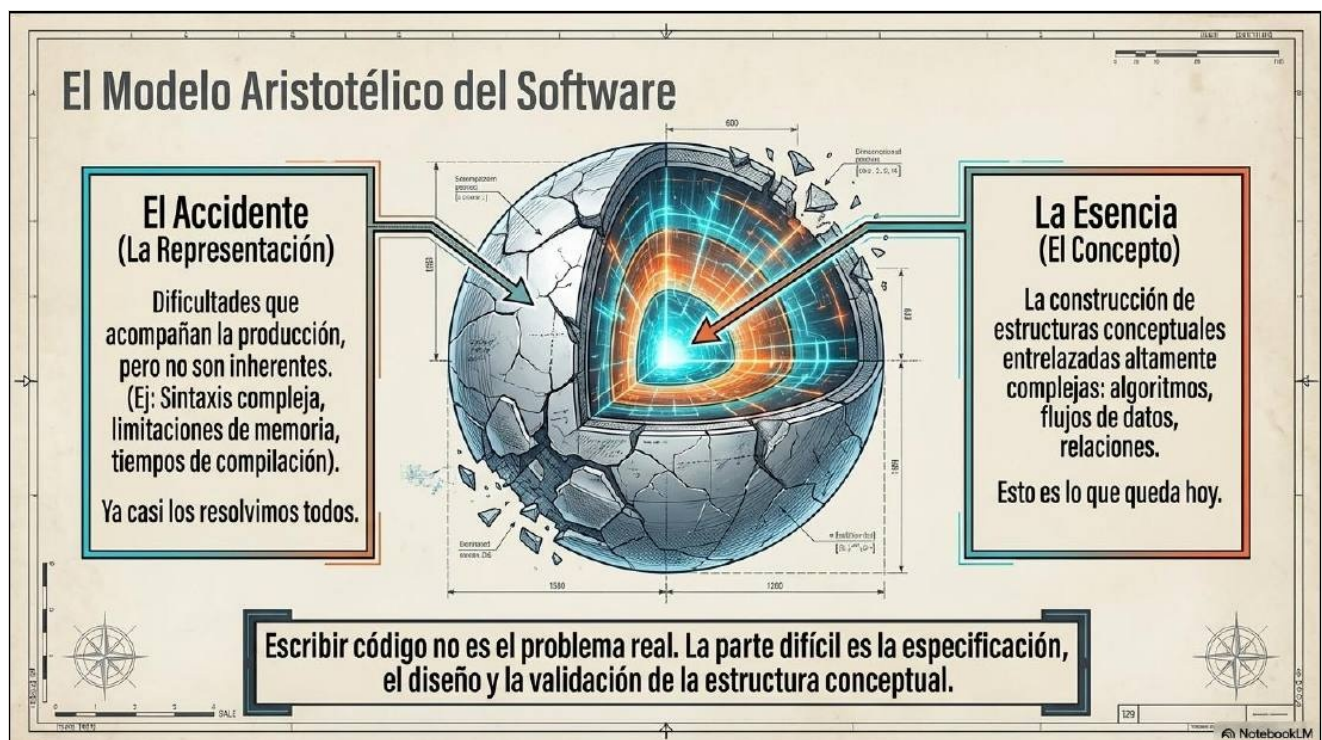


Lámina 14. El modelo aristotélico del software: lo accidental (la representación) frente a lo esencial (el concepto).

- **Dificultades accidentales:** las que acompañan a la producción de software pero no le son inherentes. Sintaxis engorrosa, limitaciones de memoria, tiempos de compilación, configurar el entorno. *Ya las resolvimos casi todas.*
- **Dificultades esenciales:** construir las estructuras conceptuales entrelazadas —algoritmos, flujos de datos, relaciones, reglas de negocio— que constituyen el sistema. *Esto es lo que queda, y es lo difícil.*

De ahí la frase que conviene memorizar: **escribir código no es el problema real; el problema es la especificación, el diseño y la validación de la estructura conceptual.**

Las cuatro propiedades irreducibles



Lámina 15. Las cuatro propiedades irreducibles: complejidad, conformidad, mutabilidad e invisibilidad.

1. **Complejidad.** Al escalar, los elementos no se repiten: crecen las interacciones de forma no lineal. En otras ingenierías se repiten piezas; en software, si una parte se repite, se la convierte en una función. Lo que queda es todo distinto.
2. **Conformidad.** El software debe adaptarse a interfaces, instituciones y sistemas humanos arbitrarios, que no siguen ningún principio unificador. La complejidad no viene de la naturaleza, viene del entorno.
3. **Mutabilidad (changeability).** Es «materia de pensamiento», infinitamente maleable, y justamente por eso todo el mundo pide cambiarlo. El software exitoso es el que más presión de cambio recibe.
4. **Invisibilidad.** No tiene representación espacial ni geométrica. Un plano de un edificio se mira; un sistema no. Se lo modela con vistas parciales (clases, despliegue, flujo), pero ninguna lo muestra entero.

Todos los grandes saltos atacaron lo accidental

La historia del software es la eliminación sistemática de lo accidental

Los mayores saltos de productividad históricos no alteraron la esencia; simplemente eliminaron barreras artificiales.



Lámina 16. Los grandes saltos de productividad atacaron siempre el esfuerzo accidental, nunca el esencial.

- **Años 60 — lenguajes de alto nivel.** Liberaron al programador de bits, registros y hardware. Brooks les atribuye el mayor salto: un factor de 5.
- **Años 70 — time-sharing.** Preservó la inmediatez: se eliminó el olvido que provocaba esperar horas por una compilación batch.
- **Años 80/90 — entornos integrados (IDEs).** Herramientas unificadas, interconexión de programas y formatos.

Ninguno de los tres tocó la esencia. Todos removieron barreras artificiales. Es la razón por la que, aun con herramientas cien veces mejores, seguimos teniendo proyectos que fracasan.

La lectura actual: la IA y el cuello de botella humano

La cátedra actualiza el ensayo a la era de los asistentes de código. El argumento se sostiene: la IA es un ataque espectacular a lo **accidental** (escribir, traducir, refactorizar, generar pruebas), pero la **esencia** sigue del lado humano.

IDEA CENTRAL

«La parte más difícil de construir software no es decirlo, sino decidir qué decir». Si quien escribe el prompt no comprende la complejidad esencial del problema, el prompt será incompleto, y la IA no deduce la intención no escrita.



Lámina 18. El embudo de la especificación: el límite no es generar código, es formalizar la intención.

De ahí la imagen del **embudo de la especificación**: el ancho de banda de ejecución de la IA es prácticamente infinito, pero el ancho de banda conceptual del humano —traducir ideas vagas, reglas difusas e integraciones heredadas a restricciones formales— es el límite físico del sistema. El flujo total sigue estrangulado en el mismo punto, por más rápido que se genere el código abajo.

Cómo atacar la esencia

Brooks proponía cuatro tácticas, y siguen vigentes:

- **Buy vs. build.** El software más barato es el que no se escribe. Integrar suele costar menos que construir, incluso cuando construir es «gratis» con IA.
- **Prototipado rápido.** Los requerimientos no se relevan, se *extraen* iterativamente. Acá la IA brilla: prototipos descartables para refinar la especificación.
- **Grow, not build.** Desarrollo incremental: hacer correr un esqueleto mínimo y engrosarlo orgánicamente, manteniendo siempre un sistema funcional.
- **Grandes diseñadores.** La diferencia entre un diseño bueno y uno grande se acerca a un orden de magnitud. Los grandes diseños vienen de grandes diseñadores, y a esos hay que buscarlos, formarlos y retenerlos.

La conclusión de la unidad, en una frase: **la IA nos dio la velocidad de la luz para recorrer el laberinto, pero no nos dio el mapa.**

07 Qué dicen los datos: éxito y fracaso

lám. 21–23

Los porcentajes provienen de los estudios sobre proyectos de software (informes CHAOS, del Standish Group) e indican el peso relativo de cada factor.

Cuando el proyecto sale bien es por...

Involucramiento del usuario	15,9 %
Apoyo de la gerencia	13,0 %

Enunciado claro de los requerimientos	9,6 %
Planeamiento adecuado	8,2 %
Expectativas realistas	7,7 %
Hitos intermedios	7,7 %
Personas involucradas competentes	7,2 %

Cuando sale mal es por...

Requerimientos incompletos	13,1 %
Falta de involucramiento del usuario	12,4 %
Falta de recursos	10,6 %
Expectativas poco realistas	9,3 %
Falta de apoyo de la gerencia	8,7 %
Requerimientos cambiantes	8,1 %

Las dos listas son casi espejo: los mismos factores, dados vuelta. Y ninguno de los primeros lugares es técnico. **Los proyectos no fracasan por falta de talento programando, fracasan por requerimientos, personas y expectativas.** De ahí el chiste de la lámina 22 —el brief del cliente muestra un tigre, el presupuesto alcanza para un gato enojado y lo que se entrega es un nene disfrazado— y de ahí la conclusión de la unidad:

CONCLUSIÓN

Saber programar no es suficiente.

08 Qué es la Ingeniería de Software

Lám. 24

Tres definiciones que conviene tener a mano, porque cada una subraya algo distinto:

Parnas

Multi-person construction of multi-version software.

Definición mínima y filosa. Lo que distingue a la ingeniería de software de «programar» son dos plurales: **varias personas** (aparece la coordinación y la comunicación) y **varias versiones** (aparece la evolución y el mantenimiento). Un programa hecho por una persona para usar una vez no necesita ingeniería.

Nota: en la filmina figura como «Parmas»; el autor es David L. Parnas.

IEEE / SWEBOK

La ingeniería de software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software.

Las tres palabras clave son **sistemático** (hay un método, no improvisación), **disciplinado** (se sigue aunque cueste) y **cuantificable** (se mide; sin medición no hay mejora).

Sommerville

Una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento.

El énfasis está en **todos los aspectos** y en el alcance temporal completo: la construcción es apenas una porción del ciclo, y por lo general la más corta.

09 SWEBOK y SEBoK: los cuerpos de conocimiento

lám. 25–33

SWEBOK — Software Engineering Body of Knowledge

Publicado por la IEEE Computer Society, es el intento de delimitar qué debe saber un ingeniero de software. La cátedra trabaja con la **versión 3.0 (2014)**, organizada en **15 áreas de conocimiento**:

1. Software Requirements	9. Software Engineering Models and Methods
2. Software Design	10. Software Quality
3. Software Construction	11. Software Engineering Professional Practice
4. Software Testing	12. Software Engineering Economics
5. Software Maintenance	13. Computing Foundations
6. Software Configuration Management	14. Mathematical Foundations
7. Software Engineering Management	15. Engineering Foundations
8. Software Engineering Process	

Vale la pena mirar la lista completa: sólo *Construction* corresponde a lo que coloquialmente se llama «programar». Las otras catorce áreas son el resto del oficio.

PARA ACTUALIZAR

La IEEE publicó posteriormente una versión 4.0 del SWEBOK, con reordenamientos y nuevas áreas (entre ellas seguridad y arquitectura). Si citas el SWEBOK en un trabajo práctico, aclarará con qué versión estás trabajando.

SEBoK — Systems Engineering Body of Knowledge

Su equivalente para la **ingeniería de sistemas** (versión 1.9.1 de 2018 en las láminas). Es más amplio: el software es un componente dentro de sistemas mayores. Se organiza en partes, y cada parte agrupa áreas de conocimiento:

PARTE	ÁREAS DE CONOCIMIENTO
Fundamentos de la Ing. de Sistemas	Fundamentos de sistemas · Enfoque de sistemas aplicado a sistemas de ingeniería · Ciencia de sistemas · Pensamiento sistémico · Representación de sistemas con modelos
Ingeniería de Sistemas y Gestión	Introducción a los procesos del ciclo de vida · Modelos de ciclo de vida · Definición del concepto · Definición del sistema · Realización del sistema · Despliegue y uso · Gestión de la IS · Gestión de vida del producto y servicio · Estándares
Aplicaciones	Sistemas de producto · de servicio · de empresa · sistemas de sistemas (SoS) · sistemas de salud
Facilitadores	Facilitando negocios y empresas · equipos · individuos

La materia dentro de ese mapa

Ingeniería y Calidad de Software recorta tres grupos de disciplinas:

TÉCNICAS	DE GESTIÓN	DE SOPORTE
Requerimientos	Planificación de proyecto	Gestión de configuración
Análisis y diseño	Monitoreo y control de proyectos	Aseguramiento de calidad
Construcción		Métricas
Prueba		
Despliegue		

10 El proceso de software

lám. 34–35

Un **proceso de software** es un conjunto estructurado de actividades para desarrollar un sistema de software. Tres precisiones de la lámina:

- Las actividades **varían** según la organización y el tipo de sistema. No hay un proceso universal.
- Debe ser **explícitamente modelado** si va a ser administrado. Lo que no está escrito no se puede discutir, medir ni mejorar.
- El proceso transforma entradas (requerimientos, ideas, tiempo) en salidas (productos y servicios) usando personas, materiales, energía, equipamiento y procedimientos.



Lámina 34. El proceso de software: entradas, recursos, actividades de trabajo y salidas.

Dos definiciones formales

Proceso: la secuencia de pasos ejecutados para un propósito dado.

IEEE

Proceso de software: un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

Sw-CMM

Y el triángulo que conviene recordar: todo proceso se apoya en **procedimientos y métodos, herramientas y equipos, y personas con habilidades, entrenamiento y motivación**. Si falla una de las tres patas, el proceso no se sostiene: comprar la herramienta sin formar a la gente es el error clásico.

11 Procesos definidos vs. empíricos

lám. 36-41

Esta distinción es la bisagra conceptual entre los enfoques clásicos y los ágiles.

Proceso definido

Inspirado en las líneas de producción. **Asume que se puede repetir el mismo proceso indefinidamente y obtener los mismos resultados**. La administración y el control provienen de la *predictibilidad*: entradas → proceso definido → salidas, con entregables, hitos, revisiones y aprobaciones en cada paso. El modelo en cascada (requerimientos, análisis, diseño, implementación, prueba, despliegue, explotación) es su representante típico.

Proceso empírico

Asume procesos complicados con variables cambiantes: al repetirlos se pueden obtener resultados diferentes. La administración y el control se ejercen mediante *inspecciones frecuentes y adaptaciones*. Funcionan bien con trabajo creativo y complejo. (¿A qué suena? A desarrollo de software.)

La analogía de la Universidad de California en Irvine explica la diferencia mejor que cualquier definición: en vez de diseñar las sendas peatonales de antemano, **sembraron pasto, esperaron un año, miraron dónde la gente había hecho «caminito» y ahí construyeron las sendas**. El primer enfoque es definido; el segundo, empírico.

Patrón de conocimiento en procesos empíricos

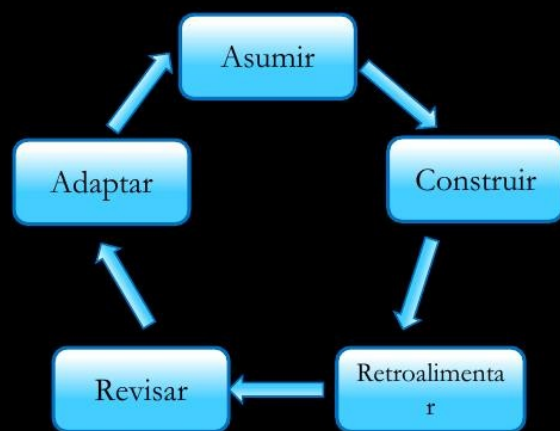


Lámina 41. El ciclo empírico y la conversión de conocimiento tácito en explícito.

El ciclo empírico es **asumir → construir → retroalimentar → revisar → adaptar**, y se repite. Su función es hacer emerger el

conocimiento tácito: lo que un individuo *sabe hacer* (habilidades) y lo que un grupo *hace en la práctica* (prácticas de trabajo) se va convirtiendo, ciclo a ciclo, en hechos explícitos y buenas prácticas compartidas.

CÓMO ELEGIR

No es «definido = malo, empírico = bueno». Cuanto mayor sea la incertidumbre sobre los requerimientos y la tecnología, más conviene un enfoque empírico. Cuanto más estable y conocido sea el dominio (y más fuertes las exigencias regulatorias o contractuales), más sentido tiene un proceso definido.

12 Ciclos de vida

lám. 42–47

Un **ciclo de vida** es la serie de pasos a través de los cuales progresa un producto o un proyecto. Hay que distinguirlos, porque no coinciden:

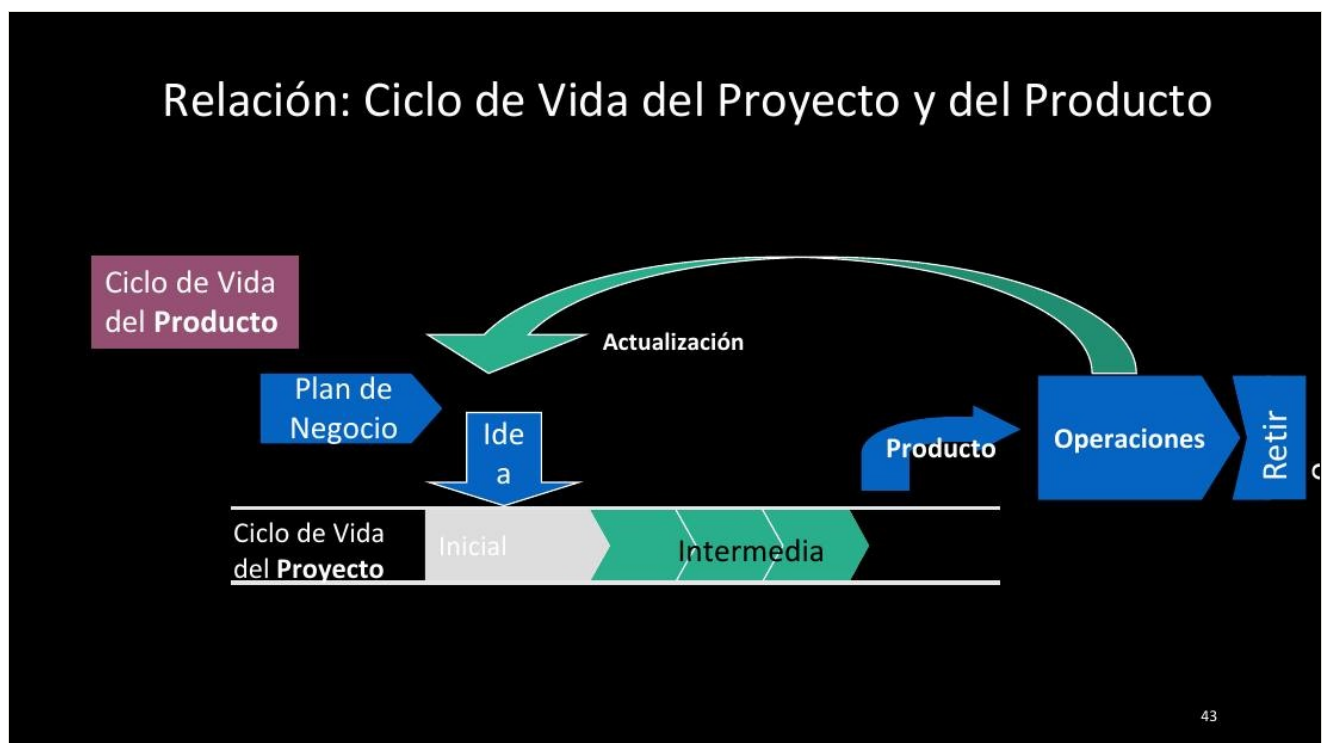


Lámina 43. El ciclo de vida del proyecto es un tramo dentro del ciclo de vida del producto.

- El **ciclo de vida del producto** arranca con la idea y el plan de negocio, y sigue con el producto en operaciones, sus actualizaciones y finalmente el retiro.
- El **ciclo de vida del proyecto** es un tramo dentro de ese arco: tiene fase inicial, intermedia y final, y termina cuando se entrega. Un mismo producto puede atravesar muchos proyectos a lo largo de su vida (cada actualización puede ser uno).

Un modelo de ciclo de vida es **una representación de un proceso**: lo describe desde una perspectiva particular y específica de cosas: las **fases** (requerimientos, especificación, diseño...) y el **orden** en que se llevan a cabo.

Clasificación

- **Secuencial**: cada fase termina antes de que empiece la siguiente (cascada y sus variantes).
- **Iterativo / incremental**: se repiten ciclos que agregan funcionalidad, con un producto utilizable en cada vuelta.
- **Recurso**: el mismo esquema se aplica en distintos niveles de descomposición del sistema.

Es la pregunta de la lámina 46. El **proceso** define *qué* actividades se hacen, con qué métodos y qué productos generan. El **ciclo de vida** define *cuándo* y *en qué orden* se organizan esas actividades en el tiempo. Un mismo proceso puede instanciarse en distintos ciclos de vida, y por eso el modelo de ciclo de vida se elige según el riesgo, la estabilidad de los requerimientos y el tipo de producto.

Para ampliar el catálogo de ciclos de vida, la cátedra remite al **capítulo 7 de *Rapid Development* (Desarrollo y gestión de proyectos informáticos), de Steve McConnell**.

13 Preguntas de repaso

autoevaluación

1. ¿Por qué se dice que la ingeniería de software nació de una crisis? ¿Qué crisis?
2. Explicá la diferencia entre dificultades esenciales y accidentales con un ejemplo propio.
3. ¿Cuáles son las cuatro propiedades esenciales del software según Brooks? Elegí una y justificá por qué ninguna herramienta la elimina.
4. Si la IA escribe el código, ¿por qué sigue habiendo un cuello de botella humano? Relacionalo con el «embudo de la especificación».
5. Enumerá tres razones por las que el software no se puede tratar como manufactura.
6. Los factores de éxito y de fracaso son casi espejo. ¿Qué implica eso para un jefe de proyecto?
7. Definí «proceso de software» y explicá por qué debe modelarse explícitamente para ser administrado.
8. Compará proceso definido y empírico. Dado un proyecto con requerimientos muy inestables, ¿cuál elegirías y por qué?
9. Diferenciá ciclo de vida del producto y ciclo de vida del proyecto. ¿Puede un producto tener varios proyectos?
0. ¿Qué relación hay entre proceso de desarrollo y ciclo de vida?

14 Bibliografía

Lám. 48

Libros

- Sommerville, Ian. *Ingeniería de Software*, 9.^a edición. Addison-Wesley, 2011.
- Brooks, Frederick P. *The Mythical Man-Month* (anniversary ed.). Addison-Wesley Longman, 1995.
- McConnell, Steve. *Rapid Development*. Microsoft Press.
- Cohn, Mike. *Agile Estimating and Planning*. Prentice Hall, 2006.
- Cohn, Mike. *User Stories Applied*. Addison-Wesley, 2004.

Papers y recursos

- Brooks, F. *No Silver Bullet: Essence and Accidents of Software Engineering* (1987).
- McConnell, S. *Orphans Preferred y Software's Ten Essentials*.
- Leffingwell, D. y Behrens, P. *A User Story Primer* (2009).
- Scrum Guide — scrumguides.org
- Manifiesto Ágil — agilemanifesto.org/iso/es/
- SWEBOOK (IEEE Computer Society) y SEBoK (BKCASE).

Apunte de estudio elaborado a partir de la presentación de cátedra. Las figuras están tomadas de las láminas originales.